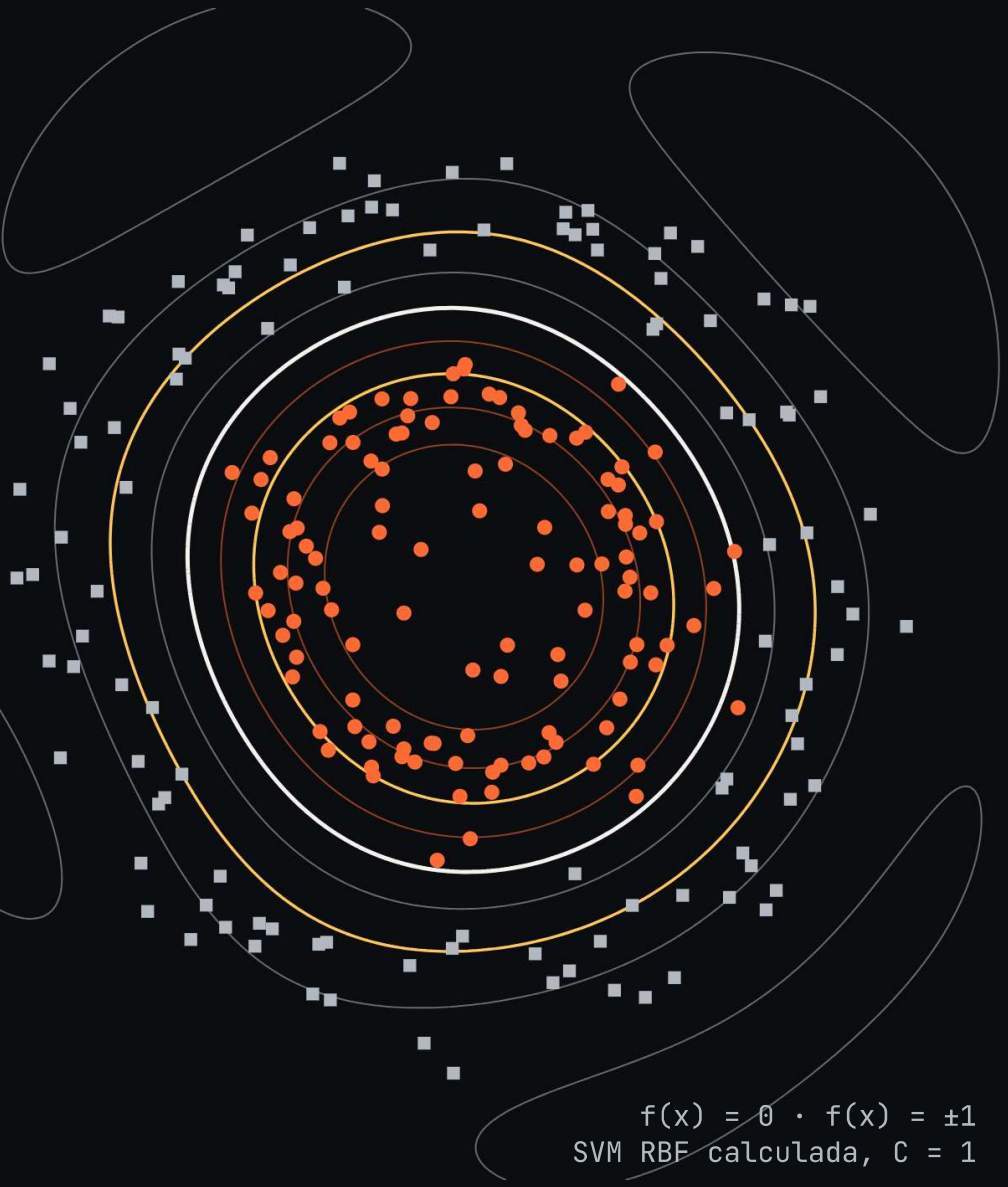
PROJECT PARALLAX · MÓDULO 7 — ALGORITMOS MODERNOS E AJUSTE FINO

AULA 7.2

SVM e o Truque do Kernel

Margem máxima, vetores de suporte e fronteiras não lineares construídas por similaridade.



120 min Teoria + exercício guiado Python · scikit-learn

f(x) = 0 · f(x) = ±1
SVM RBF calculada, C = 1

Ao final da aula, você deve conseguir explicar

Hiperplano	$w^T x + b = 0$
Margem	$2 / \|w\|$
Vetores de suporte	`support_`
Soft margin e C	ξ_i, C
Kernel e RBF	K(x, z)
Gamma e fronteira de decisão	γ

E treinar, avaliar e comparar SVM Linear e SVM RBF em Python, começando de um notebook vazio.

PLANO · 120 MIN

10′	Objetivos e recapitulação
15′	Introdução à SVM e hiperplanos
20′	Margens e vetores de suporte
15′	Hard margin, soft margin e C
20′	Kernel trick, RBF, gamma e escalonamento
30′	**Exercício guiado do zero**
10′	Quiz, erros comuns e desafio

Características, rótulos e fronteira

amostra	x_1	x_2	y
1	2,0	1,0	+1
2	1,0	2,0	-1
3	3,1	0,4	+1

Cada linha é um ponto $x = (x_1, x_2)$ no plano.

Características (x) — números que descrevem a amostra.

Rótulo (y) — a resposta a prever. Na matemática da SVM: $y \in \{-1, +1\}$. No scikit-learn pode continuar 0 e 1.

Classificação binária — apenas duas classes possíveis.

Fronteira de decisão — a curva que divide o espaço em regiões de classe.

O que já sabemos

REGRESSÃO LOGÍSTICA

$$p = \sigma(w^T x + b)$$

Probabilidade da classe positiva. Fronteira onde p = 0,5, isto é, onde $w^T x + b = 0$: uma reta.

OVERFITTING

treino ≫ validação

O modelo aprende o ruído do treino e generaliza pior para dados novos.

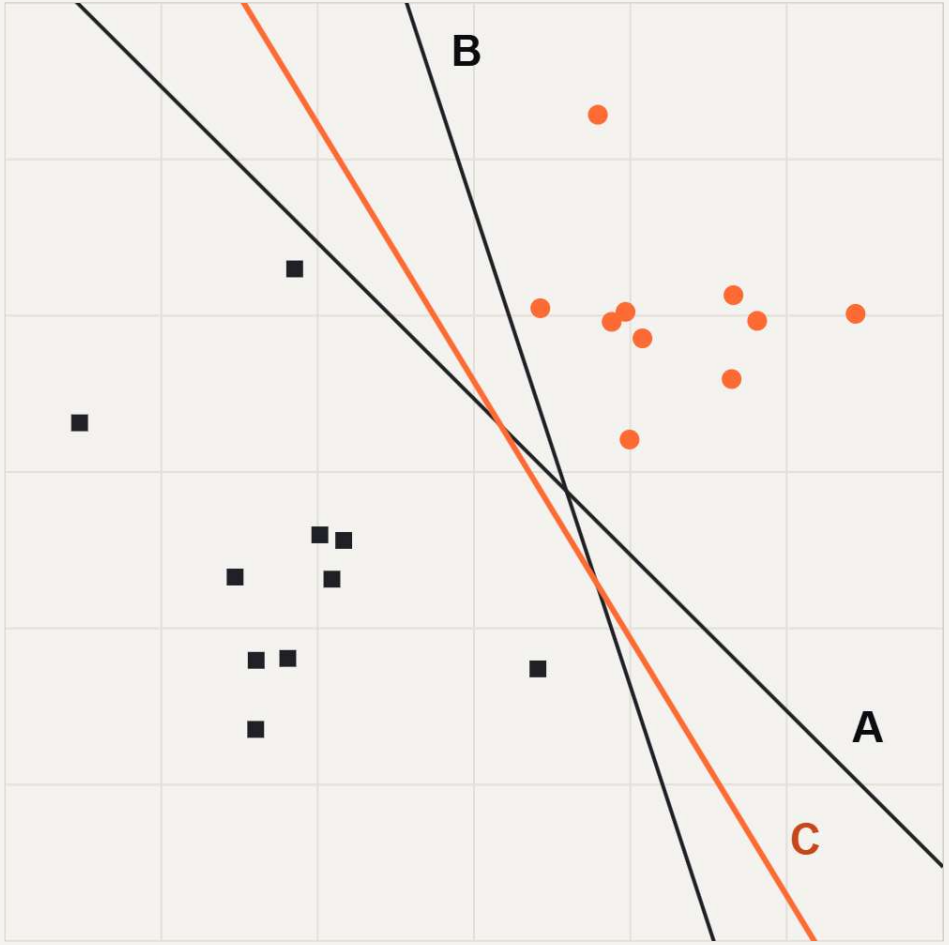
REGULARIZAÇÃO · AULA 7.1

ajuste ⇄ simplicidade

Controla o equilíbrio entre ajustar os dados e a complexidade do modelo. Na SVM, esse papel é desempenhado pelo parâmetro C.

Se várias retas separam as classes, qual delas devemos escolher?

A, B e C separam todos os pontos de treino: as três têm 100% de acurácia nesse conjunto.



● classe +1 ■ classe −1

Support

Só algumas amostras sustentam a solução. Elas são os vetores de suporte.

Vector

Cada amostra é um vetor de características: um ponto no espaço.

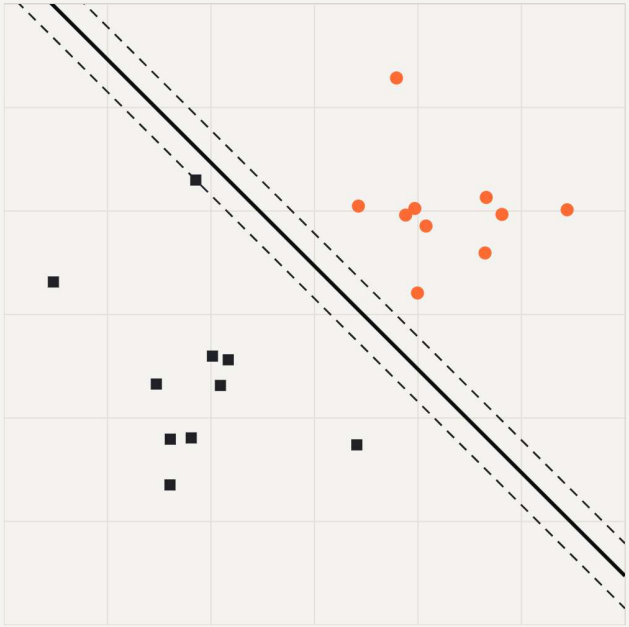
Machine

Nome histórico para algoritmo de aprendizado.

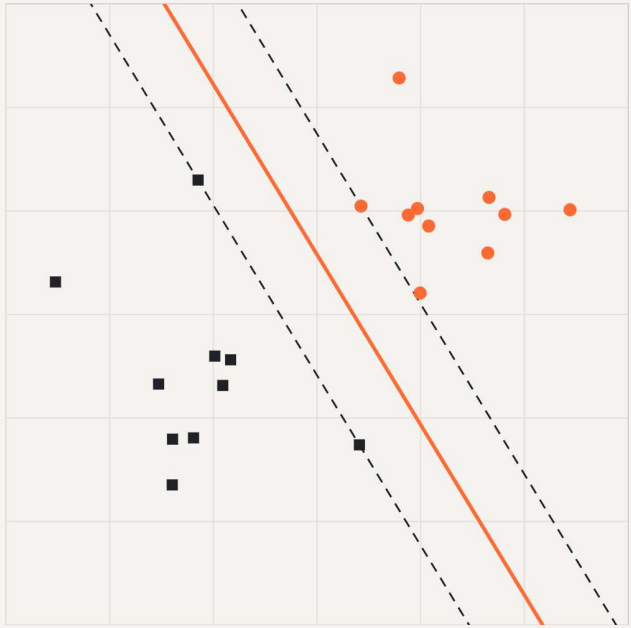
Na formulação linear, a SVM procura a fronteira que separa as classes deixando a maior faixa vazia possível entre elas: a margem.

Limite importante. Maximizar a margem é um critério bem fundamentado, mas não garante acurácia perfeita nem a melhor generalização em todo dataset.

Mesma acurácia no treino, faixas vazias muito diferentes



Reta A largura ≈ 0,44



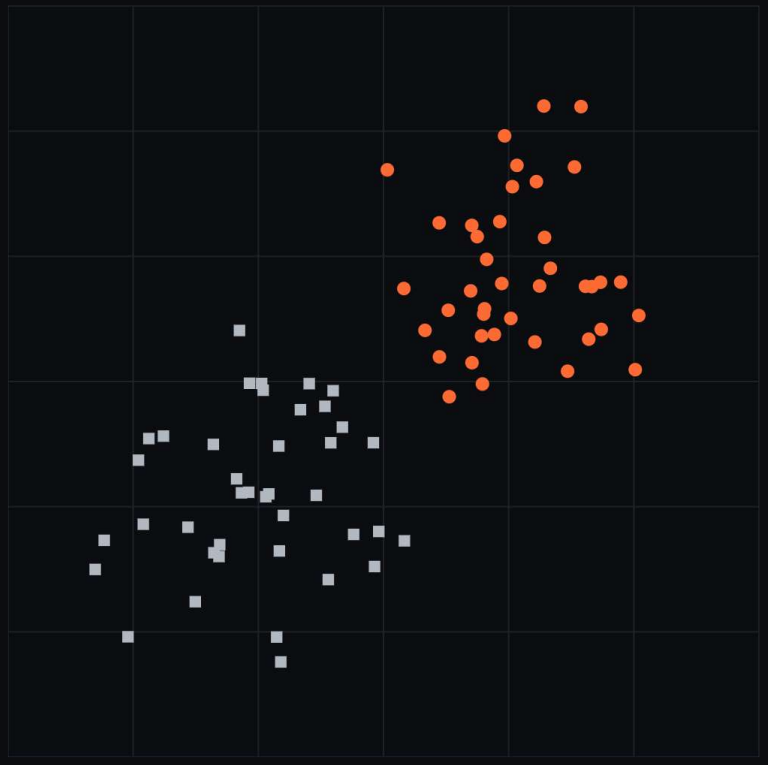
Reta C · SVM largura ≈ 1,21

As linhas tracejadas marcam a distância da reta até o ponto mais próximo, dos dois lados.

Com uma faixa larga, um ponto novo precisa se afastar mais dos exemplos de treino para cair do lado errado.

Larguras calculadas sobre o mesmo dataset sintético (semente fixa). Reta C obtida por otimização da SVM de margem rígida.

Existe uma reta que separa as classes?



Dataset A · separável por uma reta



Dataset B · círculos concêntricos

Linearmente separável: existe ao menos uma reta (ou hiperplano) com cada classe inteira de um lado.

No dataset B, toda reta deixa pontos das duas classes do mesmo lado.

Limitação estrutural de qualquer classificador linear nas características originais.

Mesma forma de fronteira, objetivos diferentes

REGRESSÃO LOGÍSTICA

minimiza a log-loss

Todos os pontos contribuem para a perda, mesmo os bem classificados e distantes da fronteira.

Saída: probabilidade, via `predict_proba()`.

SVM (MARGEM FLEXÍVEL)

maximiza a margem

Penaliza apenas pontos dentro da margem ou do lado errado; os demais contribuem com zero. A fórmula completa aparece no Bloco 9.

Saída: valor de decisão, via `decision_function()`. Não é probabilidade.

Três ferramentas de geometria

01 · VETOR

$$\mathbf{w} = (1, -1)$$

Lista ordenada de números. Pode ser lida como um ponto ou como uma seta saindo da origem.

02 · TRANSPOSTA E PRODUTO ESCALAR

$$\mathbf{w}^{\mathrm{T}}\mathbf{x} = \Sigma\, w_j\, x_j$$

^{T} transforma a coluna em linha. Linha × coluna = um número.

$(1)(2) + (-1)(1) = 1$

03 · NORMA EUCLIDIANA

$$\|\mathbf{w}\| = \sqrt{(\mathbf{w}^{\mathrm{T}}\mathbf{w})}$$

O comprimento da seta.

$\sqrt{(1^2 + (-1)^2)} = \sqrt{2} \approx 1{,}4142$

$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$

$\mathbf{x}$

Vetor de características da amostra: (x_1, x_2, ..., x_d).

$\mathbf{w}$

Vetor de pesos, um por característica. É perpendicular à fronteira: o **vetor normal**.

$\mathbf{w}^T\mathbf{x}$

Produto escalar: $w_1x_1 + w_2x_2 + \ldots$
Resultado: um número.

b

Intercepto: desloca a fronteira sem girá-la.

$f(\mathbf{x}) > 0$ lado positivo → classe +1

$f(\mathbf{x}) < 0$ lado negativo → classe −1

$f(\mathbf{x}) = 0$ **o hiperplano (fronteira)**

2 características	3 características	d características
reta	**plano**	**hiperplano**

De que lado está cada ponto?

`w = (1, -1) · b = -0,5`

`x = (2, 1)`

$$f = 1(2) + (-1)(1) - 0{,}5$$

$$f = 2 - 1 - 0{,}5 = +0{,}5$$

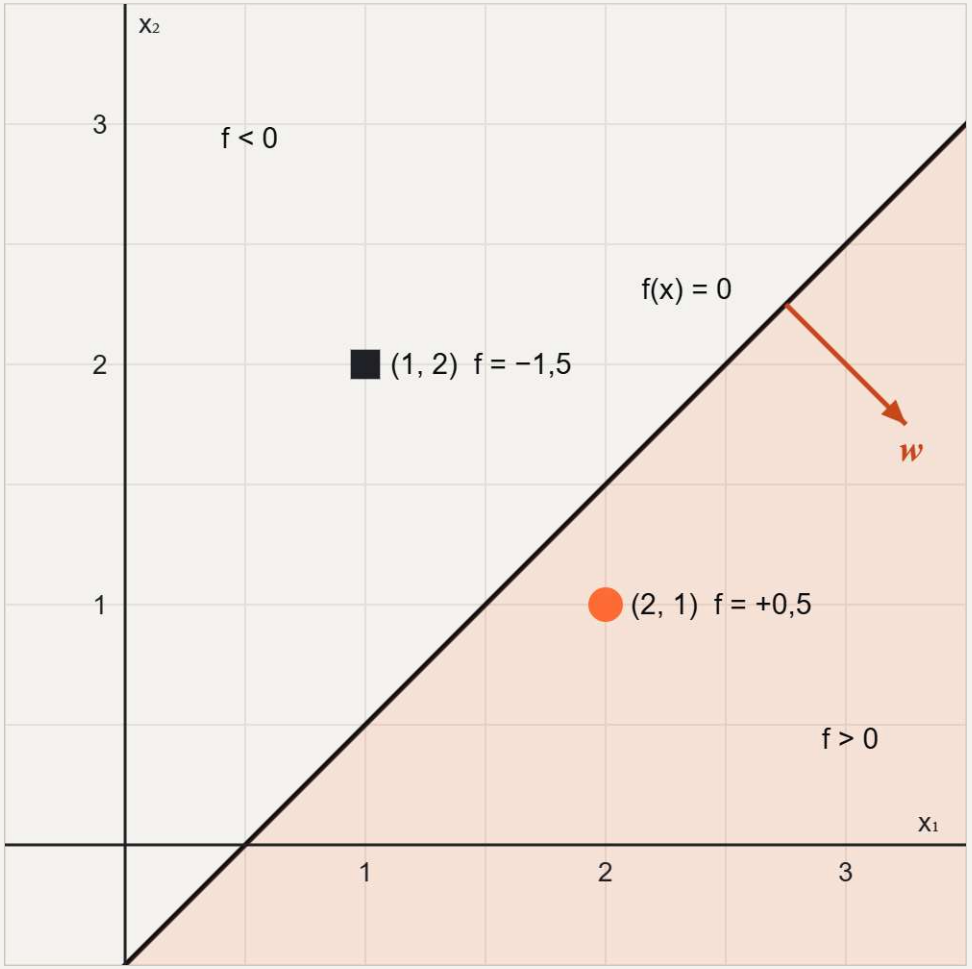
Positivo → lado positivo → classe +1

`x = (1, 2)`

$$f = 1(1) + (-1)(2) - 0{,}5 = -1{,}5$$

Negativo → lado negativo → classe –1

Fronteira: $x_1 - x_2 - 0{,}5 = 0$, isto é, $x_2 = x_1 - 0{,}5$.



$$d(\mathbf{x}, H) = \frac{|\mathbf{w}^T\mathbf{x} + b|}{\|\mathbf{w}\|}$$

H	o hiperplano $w^Tx + b = 0$
$\lvert\cdot\rvert$	valor absoluto: distância nunca é negativa; o sinal só indica o lado
$\|\mathbf{w}\|$	norma euclidiana; dividir por ela elimina o efeito da escala de w
d	distância perpendicular: o caminho mais curto até H

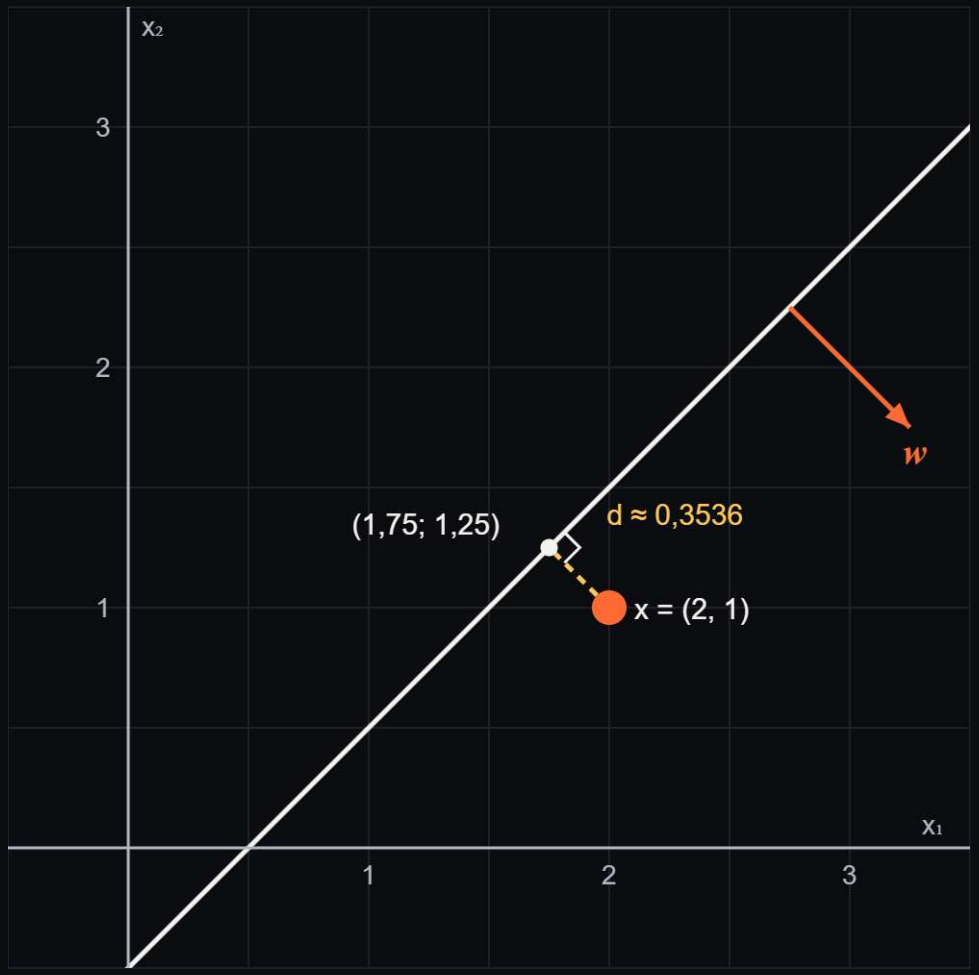
POR QUE W É PERPENDICULAR A H

$$\mathbf{w}^T\mathbf{a} + b = 0 \text{ e } \mathbf{w}^T\mathbf{c} + b = 0$$

$$\Rightarrow \mathbf{w}^T(\mathbf{a} - \mathbf{c}) = 0$$

a – c aponta ao longo de H; produto escalar zero significa ângulo reto.

$$\mathbf{x} = \mathbf{x}_p + r \cdot \mathbf{w}/\|\mathbf{w}\| \Rightarrow f(\mathbf{x}) = r\|\mathbf{w}\|$$



`w = (1, -1) · b = -0,5 · x = (2, 1)`

numerador $|f(x)| = |0{,}5| = 0{,}5$

denominador $\|w\| = \sqrt{2} \approx 1{,}4142$

distância $d = 0{,}5 / \sqrt{2} \approx 0{,}3536$

VERIFICAÇÃO GEOMÉTRICA

$$x_p = x - (f(x)/\|w\|^2)w = (2, 1) - 0{,}25 \cdot (1, -1) = (1{,}75; 1{,}25)$$

Está na reta: 1,75 – 1,25 – 0,5 = 0. Distância entre os pontos: $\sqrt{(0{,}25^2 + 0{,}25^2)} = \sqrt{0{,}125} \approx 0{,}3536$.

Margem: a distância até a amostra mais próxima

$$\text{margem} = \min_i y_i \, (\mathbf{w}^T \mathbf{x}_i + b) \, / \, \|\mathbf{w}\|$$

A MESMA RETA, INFINITAS ESCALAS

$$x_1 - x_2 - 0{,}5 = 0$$

$$2x_1 - 2x_2 - 1 = 0$$

Multiplicar w e b por k ≠ 0 mantém a reta. f é multiplicado por k, ‖w‖ também, e a distância não muda.

NORMALIZAÇÃO CANÔNICA (MARGEM RÍGIDA)

Escolhemos a escala em que as amostras mais próximas têm |f(x)| = 1. Surgem as bordas da margem:

$$\mathbf{w}^T \mathbf{x} + b = +1$$

$$\mathbf{w}^T \mathbf{x} + b = -1$$

O valor 1 é uma convenção de escala, válida para dados separáveis.

LARGURA DA MARGEM

$2 / \|\mathbf{w}\|$

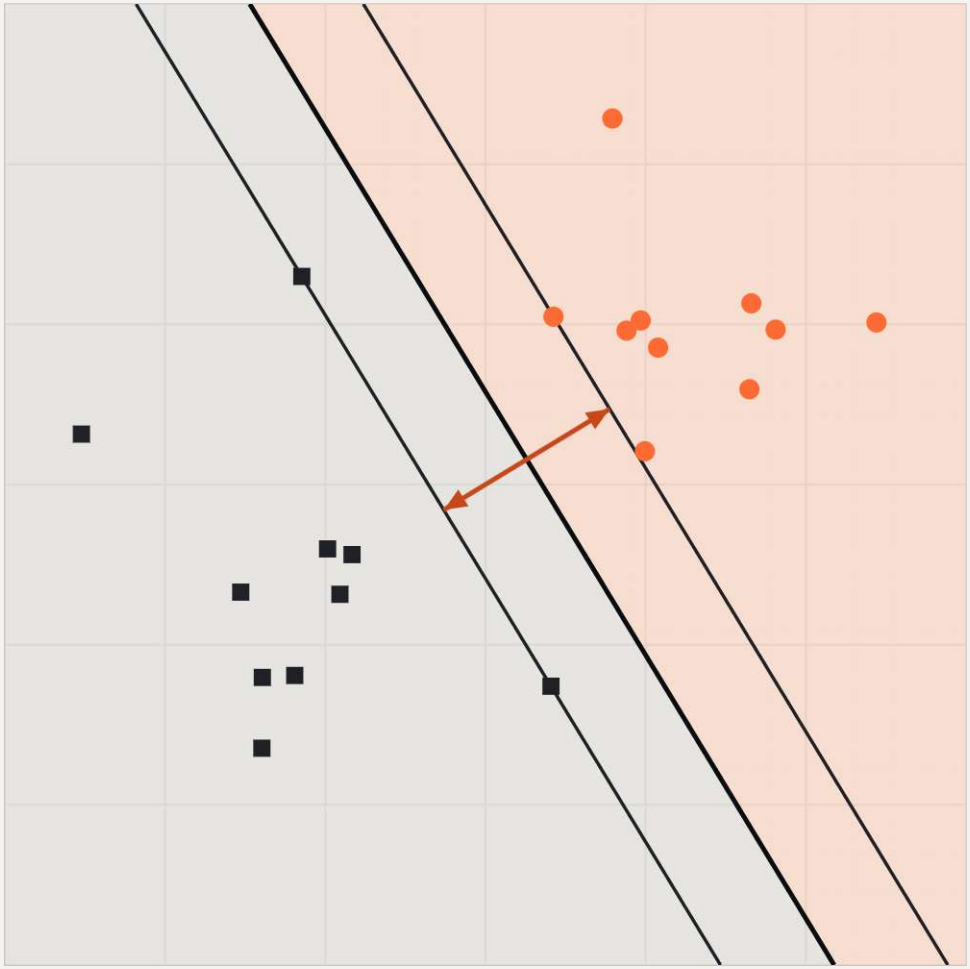
$$d(\mathbf{x}_+) = |+1| / \|\mathbf{w}\| \cdot d(\mathbf{x}_-) = |-1| / \|\mathbf{w}\|$$

$$\text{largura} = 1/\|\mathbf{w}\| + 1/\|\mathbf{w}\|$$

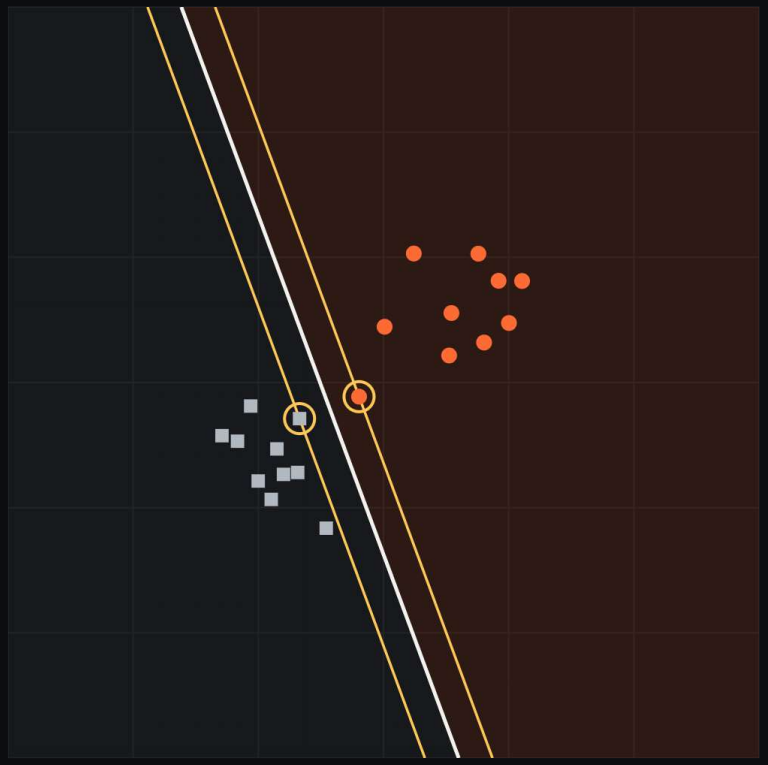
---

maximizar $2/\|w\| \Leftrightarrow$ minimizar $\|w\| \Leftrightarrow$ minimizar $\frac{1}{2}\|w\|^2$

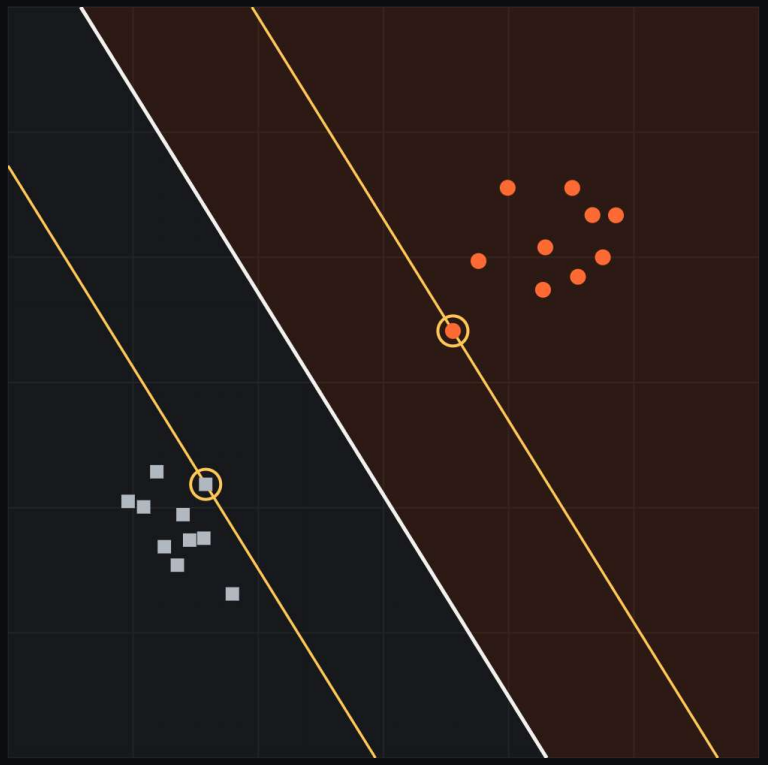
Calculado neste dataset: $\|w\| \approx 1{,}6495 \rightarrow 2/\|w\| \approx 1{,}2125$
Válido sob a normalização canônica.



‖w‖ grande, margem estreita.
‖w‖ pequeno, margem larga.



‖w‖ ≈ 3,948 2/‖w‖ ≈ 0,507



‖w‖ ≈ 0,860 2/‖w‖ ≈ 2,324

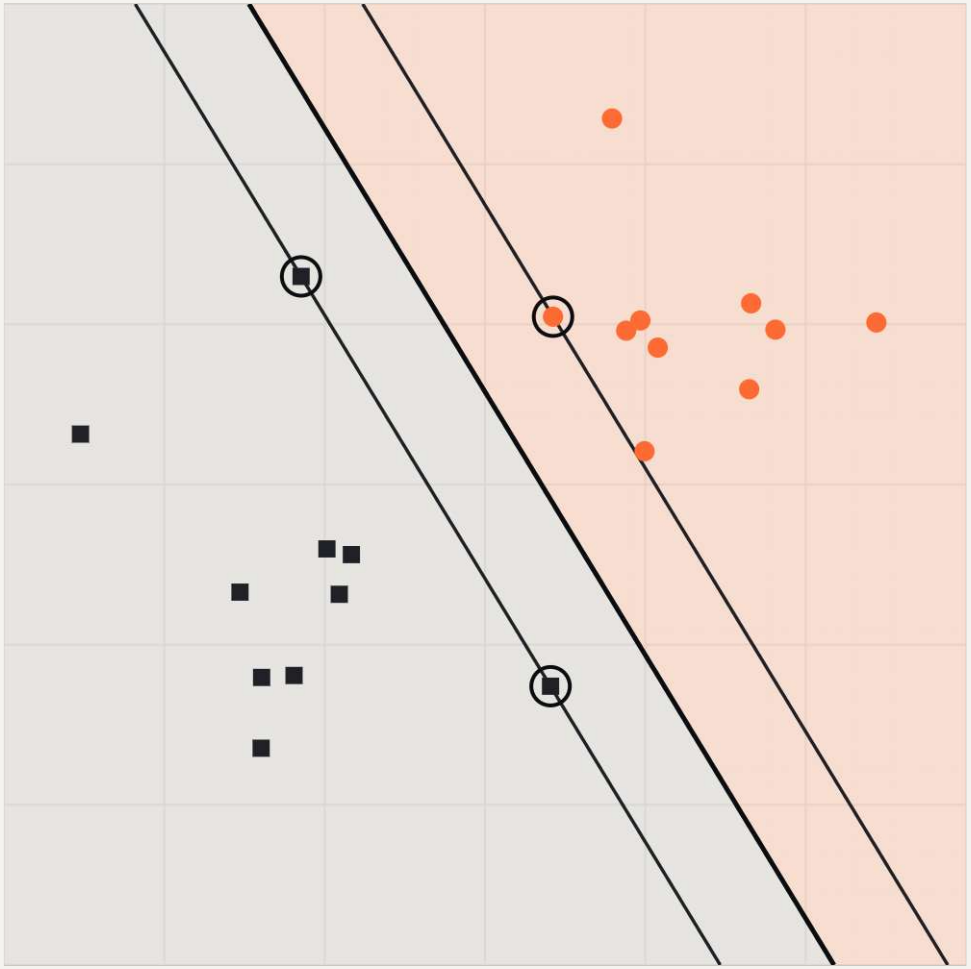
‖w‖ mede a inclinação de f. Uma f inclinada vai de −1 a +1 em pouco espaço.

— f(x) = 0

----- f(x) = ±1

○ vetor de suporte

SVMs de margem rígida calculadas em dois datasets sintéticos separáveis.



3 de 20 amostras definem a fronteira

Vetores de suporte são as amostras que desempenham papel fundamental na definição da solução da SVM.

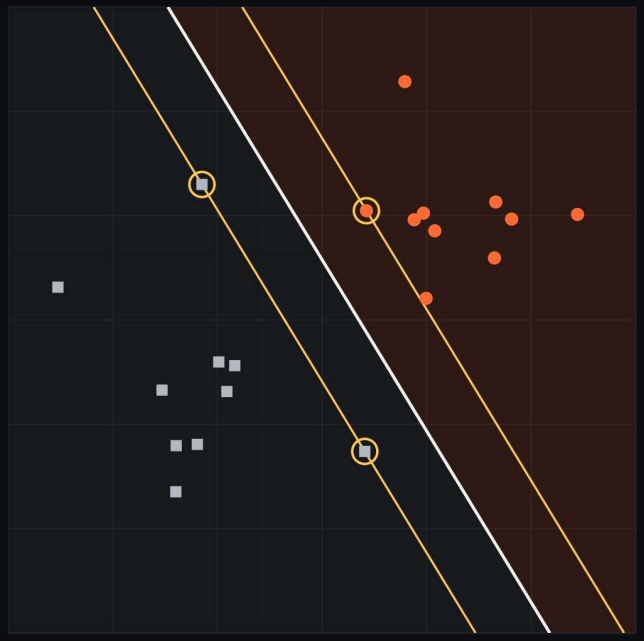
$$\mathbf{w} = \Sigma_i \, \alpha_i \, y_i \, \mathbf{x}_i, \quad \alpha_i \geq 0$$

α_i: peso de cada amostra na solução (multiplicador de Lagrange), calculado pelo otimizador. Só as amostras com $\alpha_i > 0$ entram na soma; as demais têm $\alpha_i = 0$.

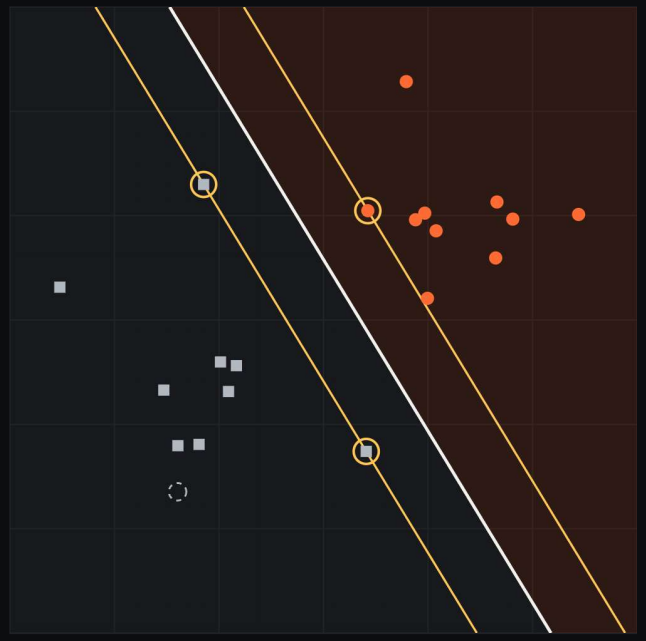
Margem rígida
sobre as bordas f(x) = ±1

Margem flexível
sobre as bordas, dentro da margem ou mal classificadas

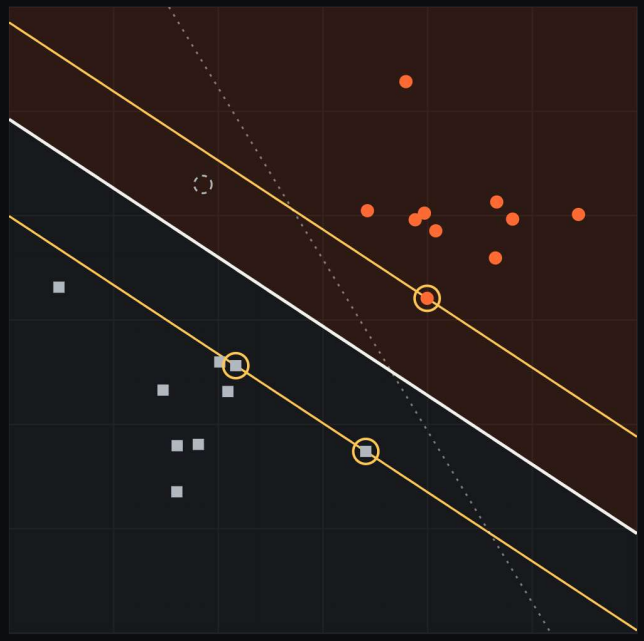
Remover um ponto comum não muda nada. Remover um vetor de suporte, sim.



1 · original
largura ≈ 1,2125



2 · sem o ponto mais distante
largura ≈ 1,2125 · inalterada



3 · sem um vetor de suporte
largura ≈ 1,547 · fronteira gira

Uma só condição para as duas classes

y_i	$f(x_i)$	$y_i \cdot f(x_i)$	lado
+1	+2	+2	correto
-1	-3	+3	correto
+1	-0,5	-0,5	errado
-1	+0,2	-0,2	errado

Com $y \in \{-1, +1\}$, o produto é positivo exatamente quando a amostra está do lado correto.

$$y_i \, f(\mathbf{x}_i) > 0$$

lado correto

$$y_i \, f(\mathbf{x}_i) \geq 1$$

lado correto **e** fora da faixa da margem (ou sobre a borda). O 1 vem da normalização canônica.

SVM DE MARGEM RÍGIDA

$$\min_{w,\,b} \; \tfrac{1}{2}\|\mathbf{w}\|^2$$

$$\text{sujeito a} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \;\; i = 1, \ldots, n$$

n, i

n amostras de treino; i indexa cada uma. Uma restrição por amostra.

$\mathbf{x}_i$, y_i

Características e rótulo (-1 ou $+1$) da amostra i. São dados, não incógnitas.

$\|\mathbf{w}\|^2$

Quadrado: tira a raiz, deixa a função suave e convexa, com o mesmo minimizador.

$\tfrac{1}{2}$

Conveniência: a derivada de $\tfrac{1}{2}\|w\|^2$ é w. Não altera a solução.

Otimização quadrática convexa. A SVC do scikit-learn resolve uma forma equivalente escrita em termos dos pesos α_i (o problema dual), com a libsvm.

Testando as restrições

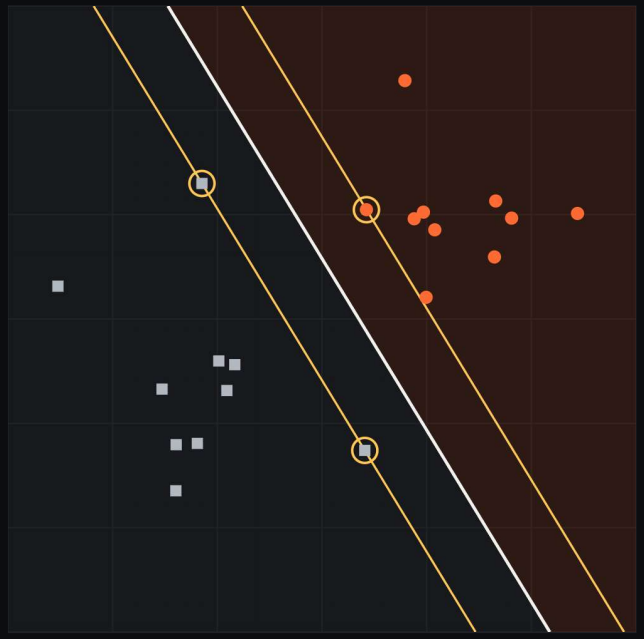
$w = (0{,}5;\ 0{,}5) \cdot b = -2 \cdot \|w\| \approx 0{,}7071$

i	x_i	y_i	$f(x_i) = 0{,}5x_1 + 0{,}5x_2 - 2$	$y_i \cdot f$	$y_i \cdot f \geq 1$?
1	(3, 3)	+1	1,5 + 1,5 - 2 = 1	1	sim, sobre a borda
2	(4, 3)	+1	2 + 1,5 - 2 = 1,5	1,5	sim
3	(1, 1)	-1	0,5 + 0,5 - 2 = -1	1	sim, sobre a borda
4	(0, 1)	-1	0 + 0,5 - 2 = -1,5	1,5	sim
5	(2,5; 2)	+1	1,25 + 1 - 2 = 0,25	0,25	**não, viola**

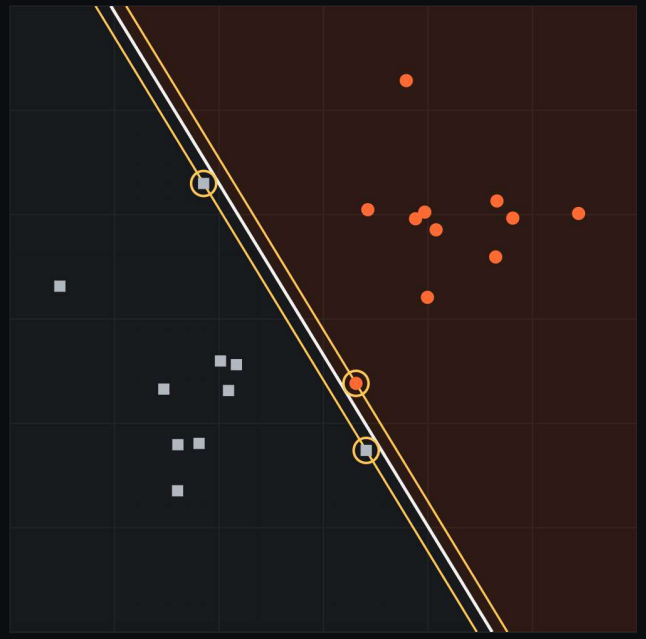
A amostra 5 está do lado correto (f > 0), mas dentro da faixa da margem. Com ela no dataset, este (w, b) não é viável para a margem rígida.

Margem deste candidato: $2/\|w\| \approx 2{,}83$. O otimizador precisaria estreitá-la até a amostra 5 ficar sobre a borda ou fora dela.

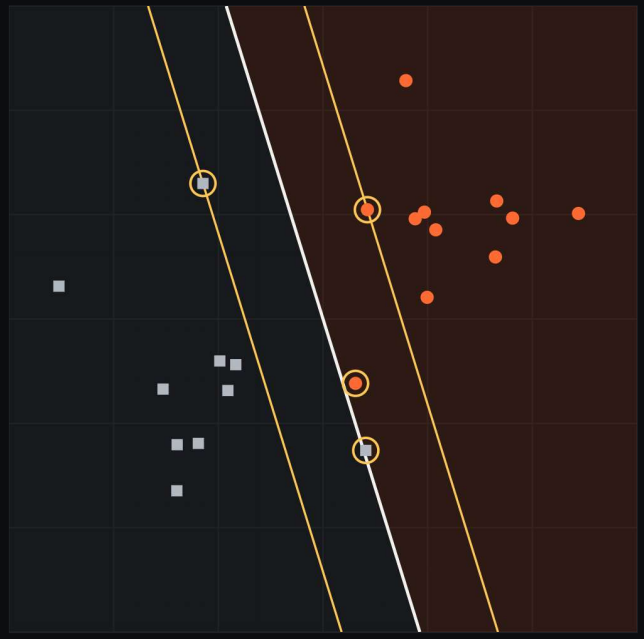
Um único outlier pode dominar a margem rígida



1 · margem rígida, sem outlier
largura ≈ 1,21



2 · margem rígida, com outlier
largura ≈ 0,25



3 · margem flexível, C = 1
largura ≈ 1,43 · treino 20/21

SVM DE MARGEM FLEXÍVEL

$$\min_{\mathbf{w},\, b,\, \xi} \; \tfrac{1}{2}\,\|\mathbf{w}\|^2 + C\,\Sigma_i\,\xi_i$$

sujeito a $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$

ξ_i (csi)

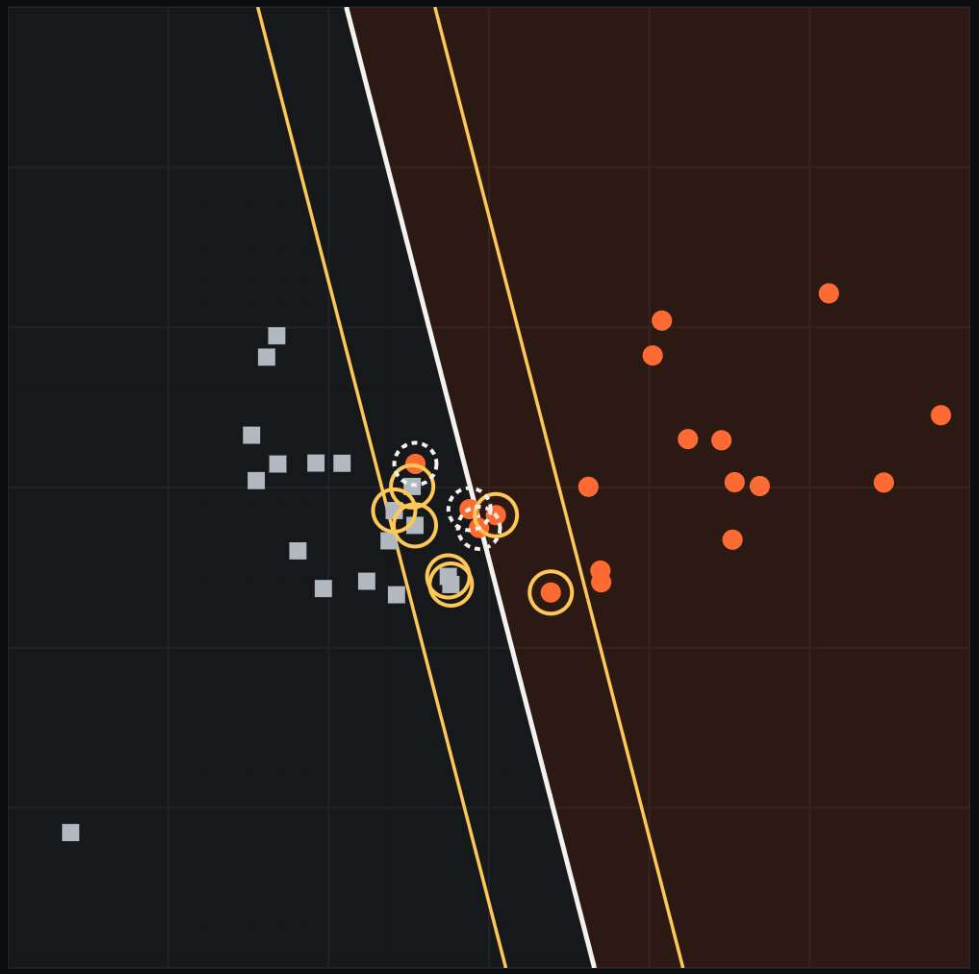
Folga da amostra i: quanto ela pode ficar aquém da borda. $\xi_i = 0$ recupera a restrição rígida.

$C\,\Sigma\,\xi_i$

Penalização total. C > 0 é o preço de cada unidade de violação: um hiperparâmetro.

$\xi_i = \max(0,\, 1 - y_i\, f(\mathbf{x}_i))$

Valor no ótimo. É a hinge loss.



Violar a margem não é o mesmo que errar

●	$\xi = 0$ lado correto, fora da margem	26
○	$0 < \xi \leq 1$ dentro da margem, ainda classificada corretamente*	7
◌	$\xi > 1$ lado errado: classificação incorreta	3

* ξ = 1: exatamente sobre f(x) = 0.
SVM linear, C = 1, 36 pontos sintéticos. 10 vetores de suporte.

C é o preço de cada violação

C pequeno

Violações baratas
Margem mais larga
Mais pontos dentro da margem
Regularização mais forte

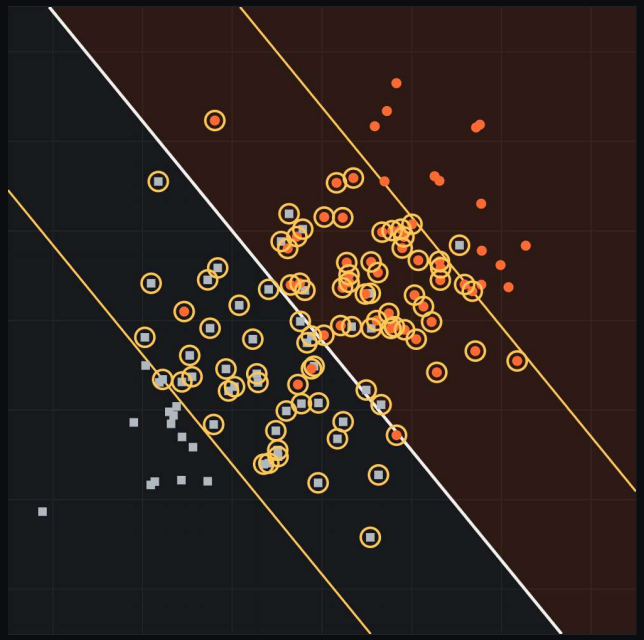
C grande

Violações caras
Margem mais estreita
Ajuste mais próximo do treino
Regularização mais fraca

C multiplica o termo de erro, não o de complexidade: funciona como o inverso da força de regularização, como o C da LogisticRegression.

O efeito real depende dos dados, do kernel e dos demais hiperparâmetros. Não existe um C bom universal.

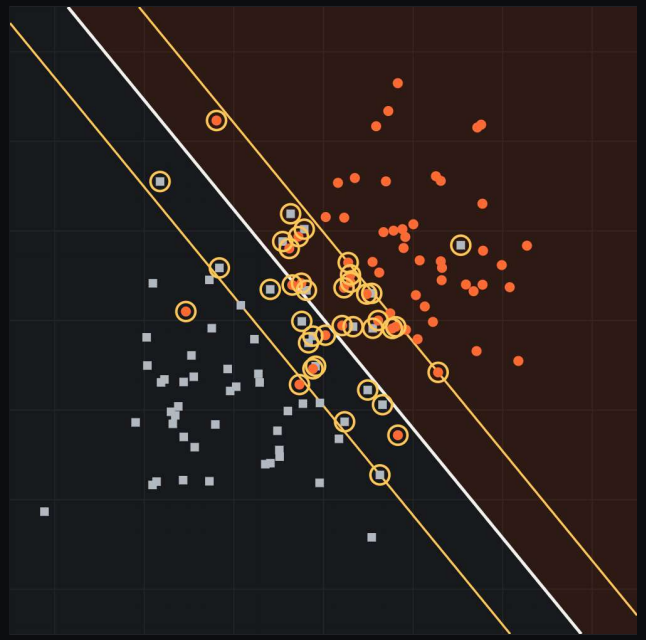
Mesmo dataset, SVM linear, três valores de C



C = 0,01

largura 3,30 · VS 92

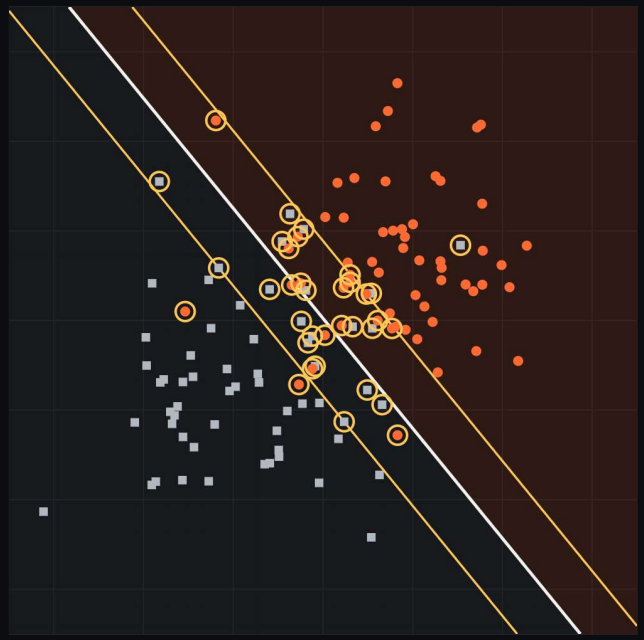
treino 88,3% · valid. 82,5%



C = 1

largura 1,23 · VS 39

treino 88,3% · valid. 86,25%

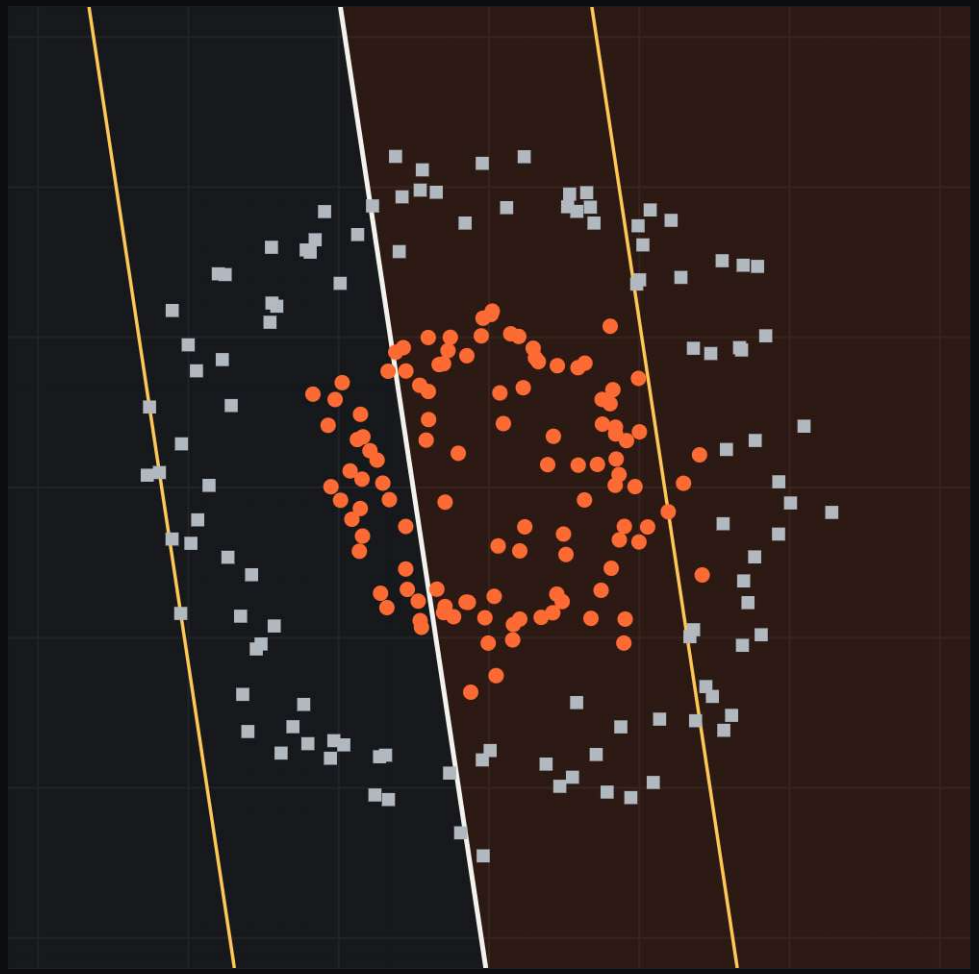


C = 100

largura 1,09 · VS 35

treino 88,3% · valid. 86,25%

Calculado: 120 pontos de treino, 80 de validação (mesma distribuição, sementes fixas). VS = vetores de suporte.



Uma reta contra dois círculos

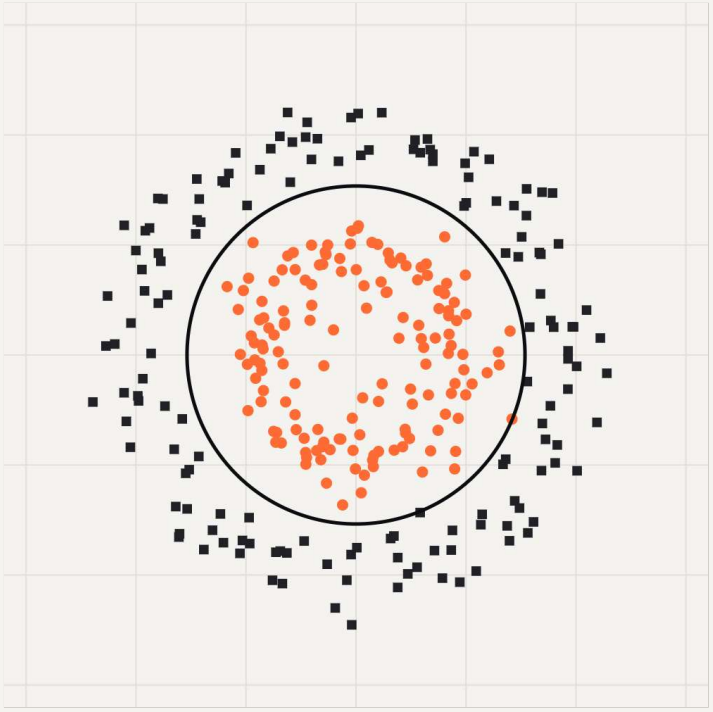
treino	58,7%
teste	49,3%
vetores de suporte	221 de 225

Não é falta de treino nem de ajuste de C. É a forma da fronteira.

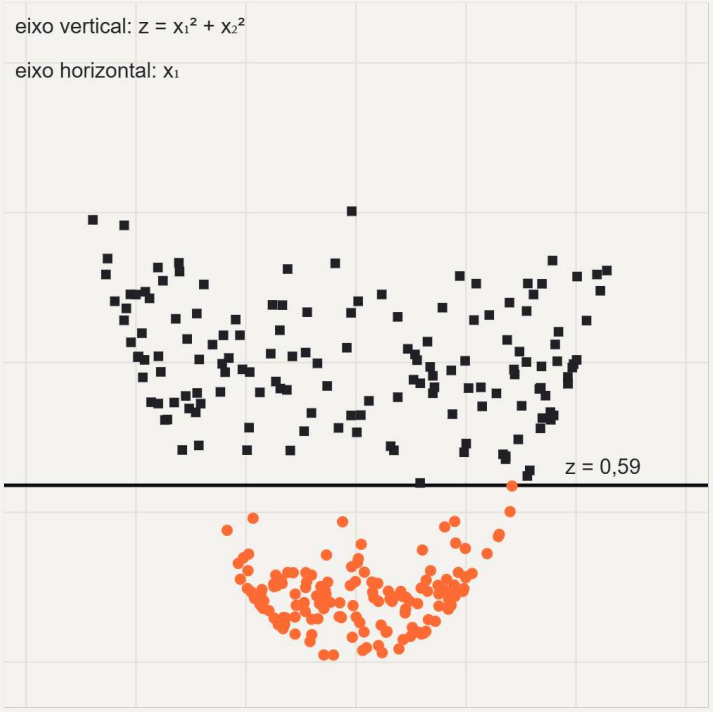
E se representássemos esses dados de outra maneira?

`make_circles(300, factor=0.45, noise=0.10, random_state=42)`, mesmo split do notebook. StandardScaler + SVC linear, C = 1. Métricas do scikit-learn 1.4.2.

Uma nova característica: $z = x_1^2 + x_2^2$



plano original: círculo $x_1^2 + x_2^2 = 0{,}59$



espaço (x_1, z): reta $z = 0{,}59$

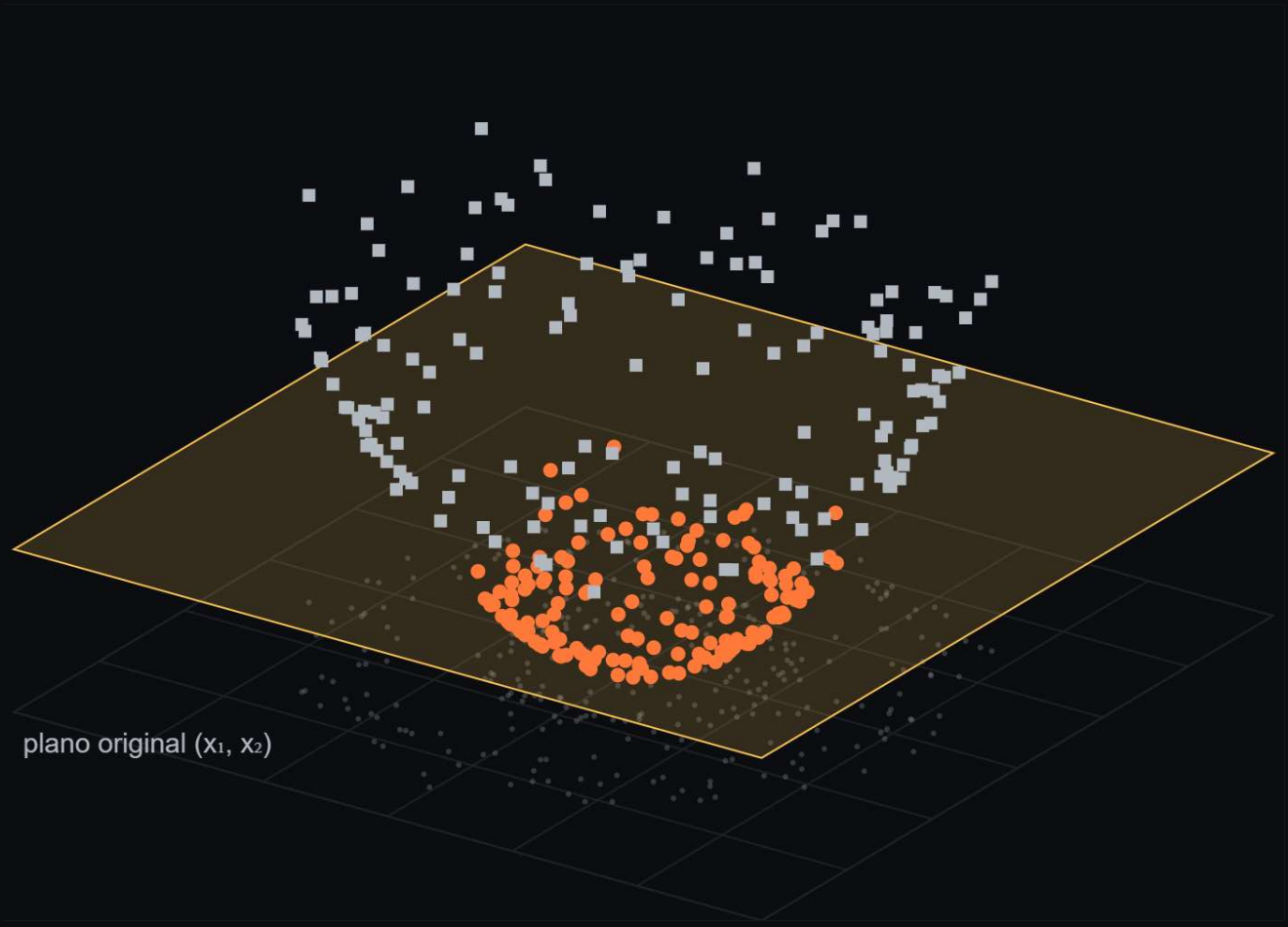
z = distância radial ao quadrado

$(0{,}3;\ 0{,}2) \rightarrow z = 0{,}13$

$(1{,}0;\ 0{,}1) \rightarrow z = 1{,}01$

Uma reta no espaço novo equivale a um círculo de raio $\sqrt{0{,}59} \approx 0{,}77$ no espaço original.

Limiar escolhido varrendo z neste dataset: separa 100% dos pontos.



Em (x_1, x_2, z), um plano separa as classes

Cada ponto sobe até a altura z. O disco fica no fundo da tigela, o anel na borda.

ATENÇÃO

Esta é uma transformação explícita e didática. O kernel RBF **não** usa essa transformação tridimensional.

Criar características explicitamente fica caro

Termos polinomiais até grau p com d características (com o termo constante):

$$C(d + p, p)$$

d	p	termos
2	2	6
100	3	176.851
1.000	3	167.668.501
RBF	—	dimensão infinita

A OBSERVAÇÃO-CHAVE

Na forma dual, a SVM só usa os dados por meio de produtos internos entre pares de amostras.

Se calcularmos esse produto diretamente, não precisamos das coordenadas transformadas.

$K(x, z) = \varphi(x)^T \varphi(z)$

x, z
duas amostras no espaço original

φ
transformação de características para um espaço, em geral, maior

K
calcula o produto interno no espaço transformado sem construir φ

CAMINHO LONGO · $\varphi(x) = (x_1^2, \sqrt{2}\, x_1 x_2, x_2^2)$

$\varphi(1, 2) = (1, 2\sqrt{2}, 4)$

$\varphi(3, 1) = (9, 3\sqrt{2}, 1)$

produto: $9 + 12 + 4 = 25$

ATALHO · $K(x, z) = (x^T z)^2$

$x^T z = 1 \cdot 3 + 2 \cdot 1 = 5$

$K = 5^2 = 25$

Mesmo número, sem calcular φ.

A previsão só precisa de similaridades com os vetores de suporte

$$f(x) = \Sigma_{i \in VS} \alpha_i y_i K(x_i, x) + b$$

$i \in VS$	só os vetores de suporte ($\alpha_i > 0$)
α_i, y_i	peso aprendido e rótulo do vetor i
$K(x_i, x)$	similaridade entre o vetor i e o ponto novo
sinal	decide a classe

RIGOR

Nem toda função é um kernel válido: K precisa ser simétrica e positiva semidefinida (condição de Mercer).

Nem todo kernel corresponde a um φ de dimensão finita. O RBF corresponde a um espaço de dimensão infinita.

KERNEL LINEAR

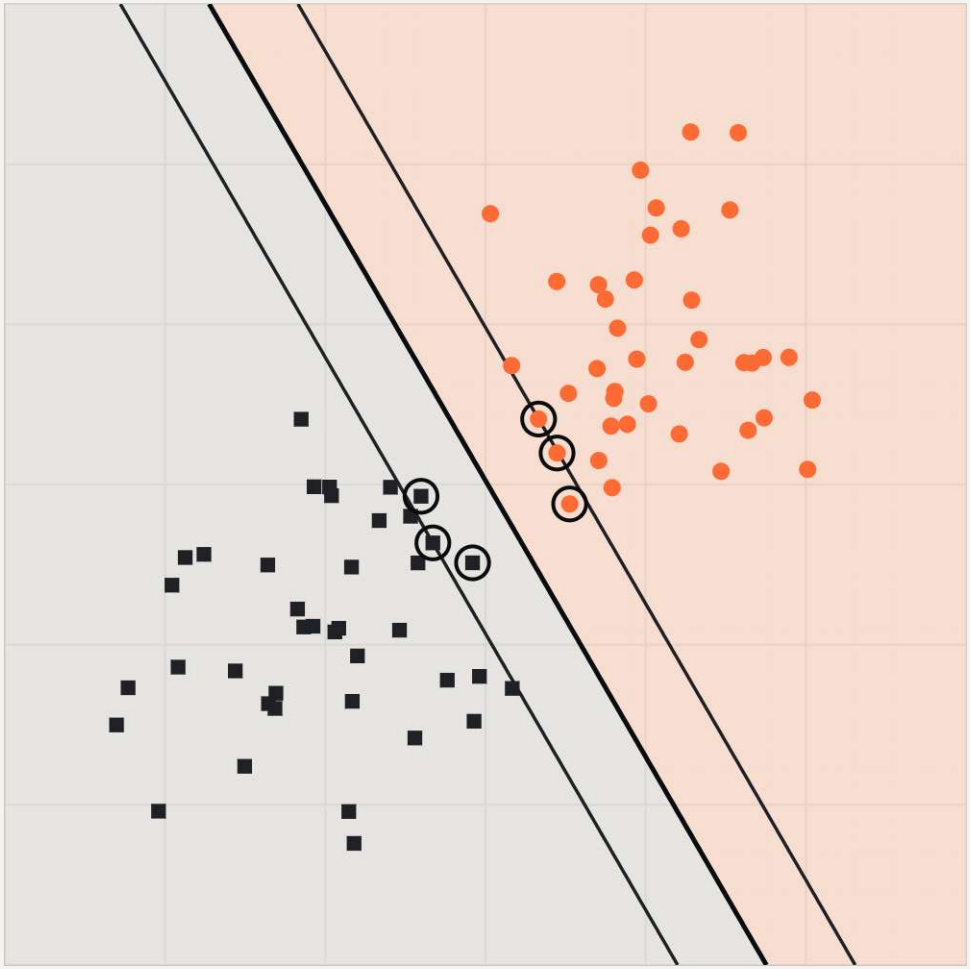
$K(x, z) = x^T z$

O produto escalar original: φ(x) = x, nenhuma característica nova.

$$f(x) = \Sigma\, \alpha_i y_i\, x_i^T x + b = w^T x + b$$

É o hiperplano do slide 11. A fronteira é sempre uma reta (ou hiperplano).

Calculado: 80 pontos, C = 1 · treino 100% · 6 vetores de suporte



$K(x, z) = \exp(-\gamma \|x - z\|^2)$

x, z

Duas amostras no espaço de características.

$\|x - z\|^2$

Distância euclidiana ao quadrado: $\Sigma (x_j - z_j)^2$.

$\gamma > 0$

Controla a rapidez com que a similaridade cai com a distância.

$\exp(-\cdot)$

Transforma distância em similaridade entre 0 e 1.

$x = z$	$K = \exp(0) = 1$ · similaridade máxima
distância ↑	$K \to 0$, nunca negativo

Cada vetor de suporte cria uma região de influência ao redor de si. A fronteira nasce da soma dessas influências.

Mesma distância, gammas diferentes

$x = (0, 0) \cdot z = (1, 0)$

$$\|x - z\|^2 = (0 - 1)^2 + (0 - 0)^2 = 1$$

γ = 0,5

$$K = \exp(-0{,}5)$$

$$\approx 0{,}6065$$

Similaridade alta: influência ampla.

γ = 2

$$K = \exp(-2)$$

$$\approx 0{,}1353$$

Similaridade baixa: influência localizada.

γ = 0,5 · z = (2, 0)

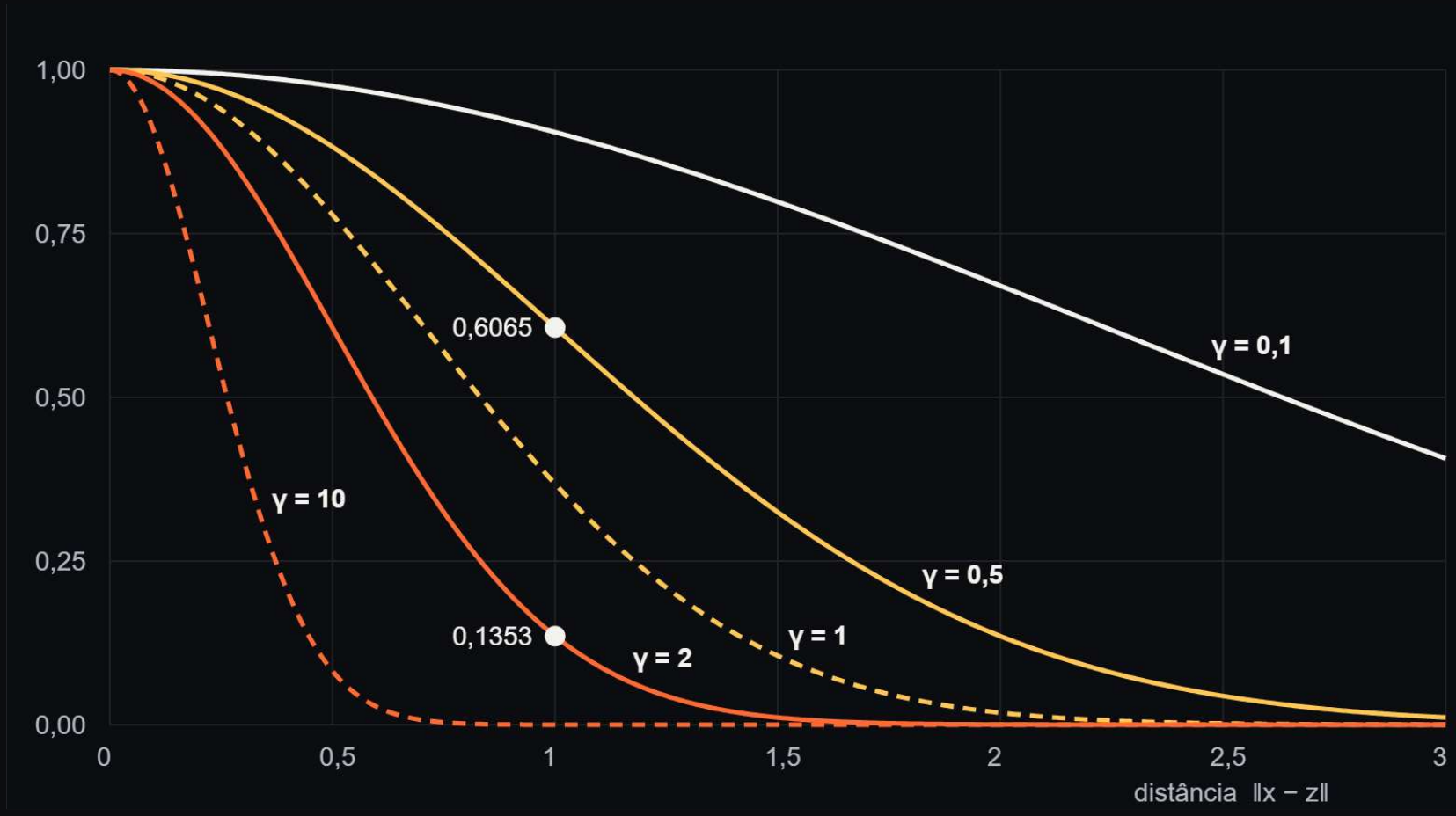
$$\|x - z\|^2 = 4$$

$$K = \exp(-2) \approx 0{,}1353$$

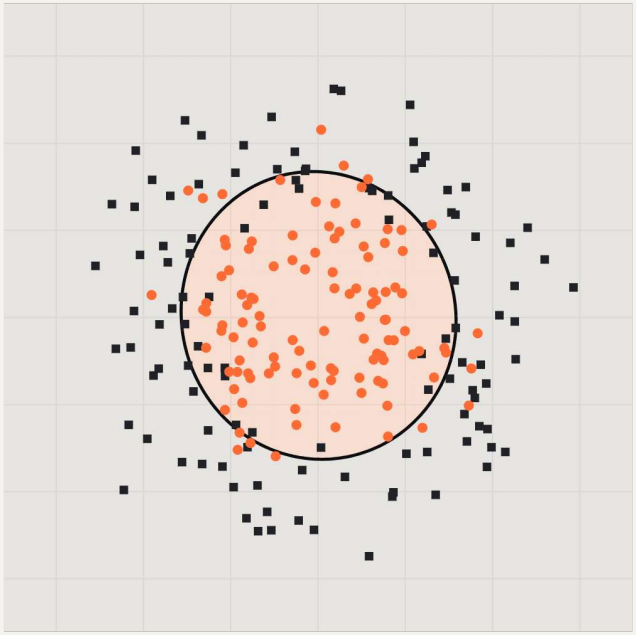
O que importa é γ · ‖x – z‖². A escala das características altera o efeito de γ.

K = exp(−γd²) para vários valores de γ

eixo vertical: similaridade K

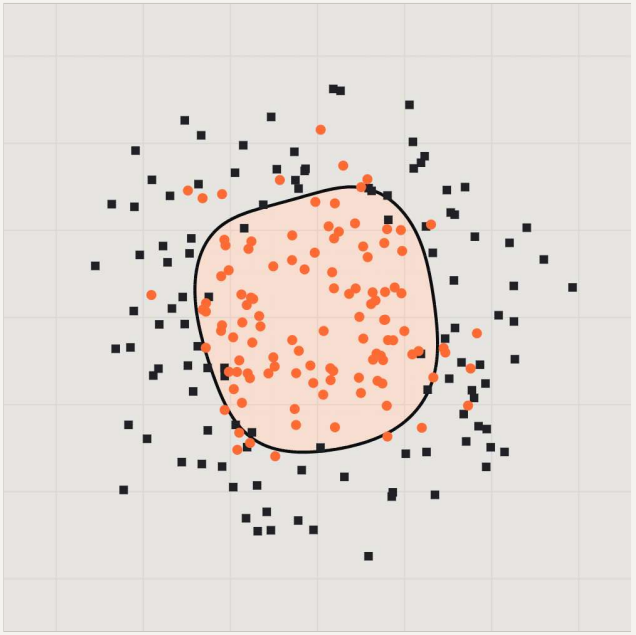


C = 1 fixo, só γ muda



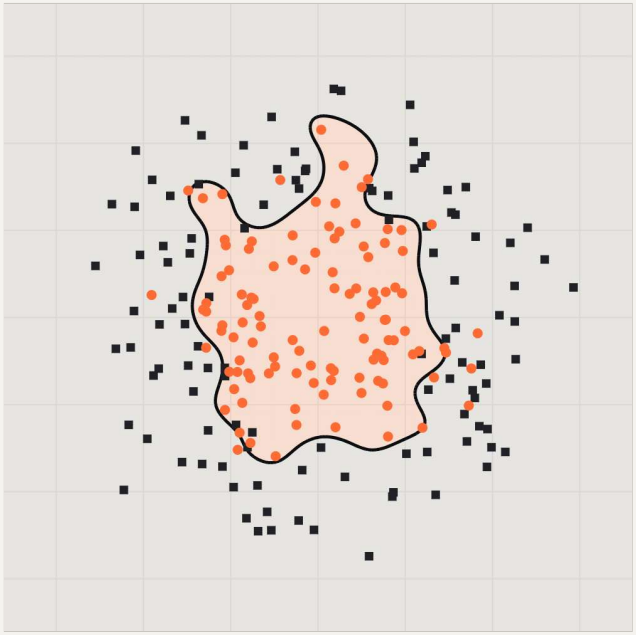
γ = 0,1 · suave

treino 82,6% · teste 89,5% · VS 142



γ = 1 · ≈ circular

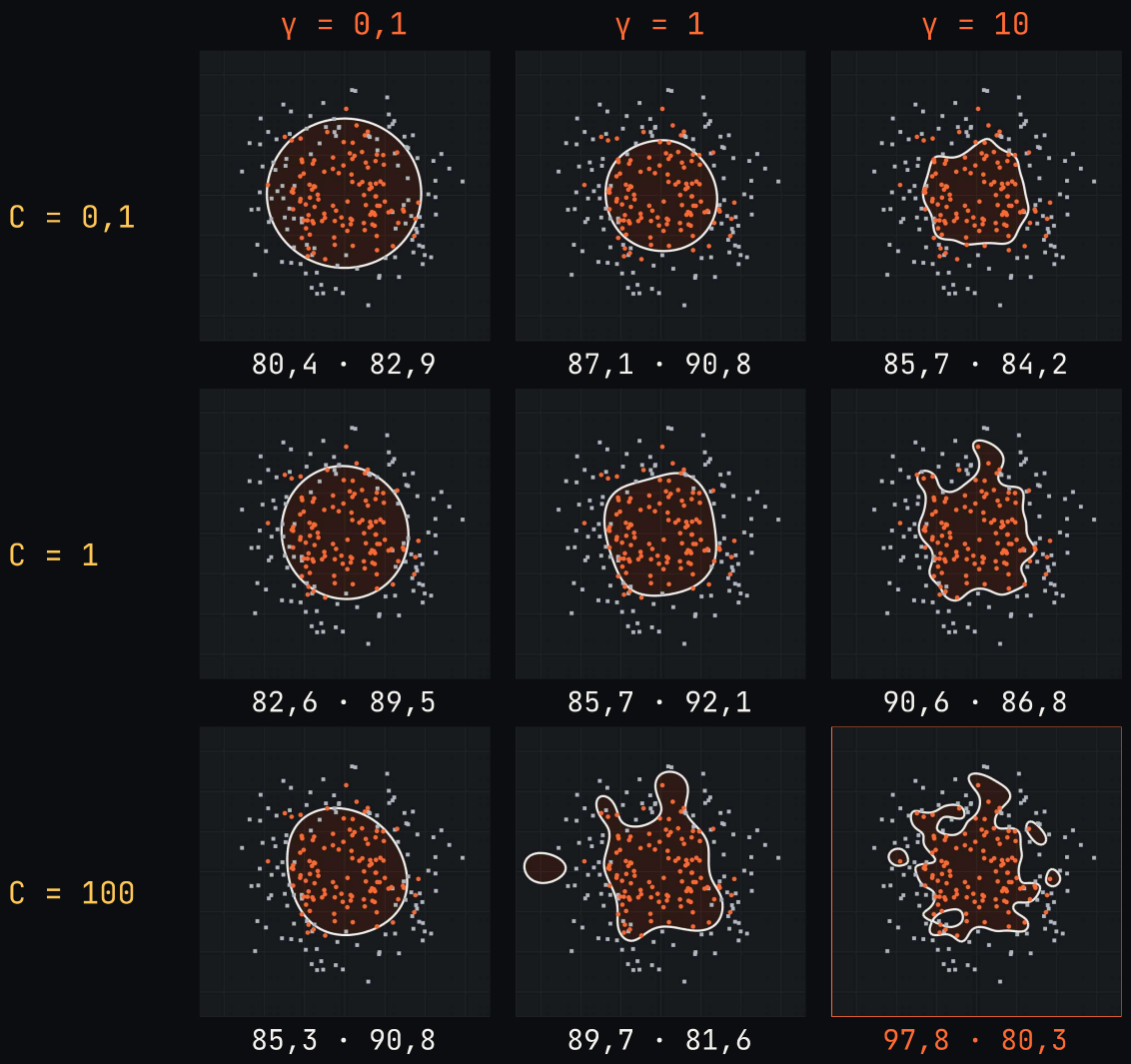
treino 85,7% · teste 92,1% · VS 104



γ = 10 · irregular

treino 90,6% · teste 86,8% · VS 167

Calculado: círculos mais ruidosos (factor 0,5, ruído 0,22), 224 treino / 76 teste, StandardScaler + SVC RBF.



C e γ devem ser escolhidos juntos

C altera o custo das violações. γ altera a localidade do kernel. Juntos, definem a complexidade da fronteira.

C = 100, γ = 10: treino 97,8%, teste 80,3%. Sobreajuste.

Célula: acurácia treino · teste (%).
Mesmo dataset e split do slide 38.

Distâncias dependem das unidades

pessoa	idade (18–70)	renda (1.000–20.000)
A	25	3.000
B	60	3.100
C	26	5.000

SEM ESCALONAMENTO

$$d(A, B) = \sqrt{(35^2 + 100^2)} \approx 105{,}9$$

$$d(A, C) = \sqrt{(1^2 + 2.000^2)} \approx 2.000$$

B parece mais próximo de A, apesar de 35 anos de diferença. A renda domina pelo tamanho dos números.

DIVIDINDO POR σ (15 ANOS · 5.000)

$$d(A, B) = \sqrt{(2{,}33^2 + 0{,}02^2)} \approx 2{,}33$$

$$d(A, C) = \sqrt{(0{,}067^2 + 0{,}4^2)} \approx 0{,}41$$

A conclusão se inverte. Para o RBF isso é decisivo: γ multiplica a distância ao quadrado.

```
from sklearn.preprocessing import StandardScaler
```

$$z = \frac{x - \mu}{\sigma}$$

μ — média no treino. Subtrair = **centralização** (média 0)

σ — desvio padrão no treino. Dividir = **padronização** (desvio 1)

EXEMPLO · IDADES 20, 30, 40

$$\mu = 30$$

$$\sigma = \sqrt{((100 + 0 + 100)/3)} \approx 8{,}165$$

$$z = -1{,}2247 \cdot 0 \cdot +1{,}2247$$

O scikit-learn usa o desvio populacional (divide por n).

`fit` calcula μ e σ: **somente no treino**

`transform` aplica a fórmula a treino, teste e dados novos

Escalonar dentro do pipeline, depois de dividir

✕ VAZAMENTO DE DADOS

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # usa o teste!

X_train, X_test, y_train, y_test = \
    train_test_split(X_scaled, y, ...)

svc = SVC(kernel="rbf")
svc.fit(X_train, y_train)
```

✓ CORRETO

```
from sklearn.pipeline import Pipeline

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, ...)

modelo = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf")),
])
modelo.fit(X_train, y_train)  # scaler: só treino
modelo.predict(X_test)        # só transform
```

Escolha C e γ na validação, não no teste



Treino ajusta w, b e os α_i

Validação compara combinações de C e γ (ex.: validação cruzada em k partes)

Teste usado uma vez, no final, para estimar o desempenho

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "svc__C": [0.1, 1, 10, 100],
    "svc__gamma": [0.01, 0.1, 1, 10],
}
busca = GridSearchCV(modelo, param_grid, cv=5)
busca.fit(X_train, y_train)
print(busca.best_params_)
print(busca.score(X_test, y_test))
# svc__C = parâmetro C do passo "svc"
# extensão do desafio, não do exercício
```

A interface da SVC

```
from sklearn.svm import SVC
```

PARÂMETROS

`kernel`
"linear", "rbf" (padrão), "poly", "sigmoid"

`C`
custo das violações (padrão 1.0)

`gamma`
número > 0, "scale" (padrão) ou "auto".
"scale" = 1 / (n_features · X.var())

MÉTODOS

`fit(X, y)`
resolve a otimização

`predict(X)`
classe prevista: um rótulo

`decision_function(X)`
valor de f(x): número real, **não é probabilidade**. f > 0 → classes_[1]

ATRIBUTOS APÓS O FIT

`support_`
índices dos vetores de suporte nos dados do fit

`support_vectors_`
coordenadas como a SVC as recebeu (escalonadas, no pipeline)

`n_support_`
quantidade por classe

Quiz rápido

01 O que é SVM?

02 O que é um hiperplano?

03 O que representa a margem?

04 O que é margem máxima?

05 O que são vetores de suporte?

06 Qual a diferença entre hard margin e soft margin?

07 O que controla C?

08 O que é kernel trick?

09 O que representa gamma?

10 Qual a diferença entre kernel linear e RBF?

11 Por que usamos escalonamento?

12 O que decision_function() retorna?

13 Por que utilizamos um Pipeline?

14 O que acontece ao separar círculos concêntricos com uma reta?

Respostas completas nas notas do professor.

EXERCÍCIO GUIADO · 30 MIN

SVM Linear vs. SVM RBF

Comece em um notebook **totalmente vazio**. Todo import, variável e função é escrito aqui.

`Aula_7_2_Exercicio_Guiado_SVM.ipynb`

Antes de entregar: Restart Kernel → Run All

01	Imports
02–03	Criar e visualizar o dataset
04	Separar treino e teste
05–09	Pipeline linear: treinar, prever, acurácia, matriz de confusão
10	Função de visualização da fronteira
11–13	Pipeline RBF: treinar e avaliar
14	**Comparar as fronteiras lado a lado**
15	Vetores de suporte nas coordenadas corretas
16–17	Experimentos com C e gamma
18–19	Tabela comparativa e conclusão

01 · IMPORTS · 02 · DATASET

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score,
                             confusion_matrix)

X, y = make_circles(
    n_samples=300,
    factor=0.45,
    noise=0.10,
    random_state=42
)
print(X.shape)
print(y.shape)
print(np.unique(y, return_counts=True))
# (300, 2)  (300,)  ([0, 1], [150, 150])
```

03 · VISUALIZAR · 04 · TREINO / TESTE

```
plt.figure(figsize=(6, 6))
plt.scatter(X[y == 0, 0], X[y == 0, 1],
            c="#B2B9C1", marker="s",
            edgecolor="k", label="Classe 0")
plt.scatter(X[y == 1, 0], X[y == 1, 1],
            c="#FF6B35", marker="o",
            edgecolor="k", label="Classe 1")
plt.xlabel("$x_1$"); plt.ylabel("$x_2$")
plt.legend(loc="upper center", ncol=2,
           bbox_to_anchor=(0.5, -0.1), frameon=False)
plt.axis("equal"); plt.show()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)
print(X_train.shape, X_test.shape)
# (225, 2) (75, 2)
```

05 PIPELINE · 06 FIT · 07 PREDICT · 08 ACURÁCIA · 09 MATRIZ

```
svm_linear = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="linear", C=1.0))
])

svm_linear.fit(X_train, y_train)

pred_linear = svm_linear.predict(X_test)
print("Reais:    ", y_test[:15])
print("Previstos:", pred_linear[:15])

acc_linear = accuracy_score(y_test, pred_linear)
print(f"Acurácia (linear): {acc_linear:.3f}")

cm_linear = confusion_matrix(y_test, pred_linear)
print(cm_linear)
```

`fit` ajusta o scaler em X_train, transforma e treina a SVC.

`predict` transforma X_test com μ e σ do treino e devolve rótulos 0 ou 1.

`acc_linear` na execução de referência: **0,493** (próximo do acaso). Matriz: [[14, 24], [14, 23]].

LEITURA DE confusion_matrix

	prev. 0	prev. 1
real 0	VN [0,0]	FP [0,1]
real 1	FN [1,0]	VP [1,1]

Diagonal = acertos. Fora dela = erros.

10 · FUNÇÃO AUXILIAR, DEFINIDA AQUI · A GRADE USA AS COORDENADAS ORIGINAIS

```
def plot_decision_boundary(model, X, y, ax, title,
                           x_lim, y_lim,
                           resolution=300):
    # 1. grade em coordenadas ORIGINAIS
    xx, yy = np.meshgrid(
        np.linspace(x_lim[0], x_lim[1], resolution),
        np.linspace(y_lim[0], y_lim[1], resolution),
    )
    grid = np.c_[xx.ravel(), yy.ravel()]

    # 2. o pipeline escalona a grade sozinho
    Z = model.predict(grid).reshape(xx.shape)
    D = model.decision_function(grid).reshape(
        xx.shape)

    # 3. regiões, fronteira (0) e margens (±1)
    ax.contourf(xx, yy, Z, levels=[-0.5, 0.5, 1.5],
                colors=["#E7E5E1", "#F8DED1"])
    ax.contour(xx, yy, D, levels=[-1, 0, 1],
               colors="k",
               linestyles=["--", "-", "--"])
```

```
    # 4. amostras (continuação da função)
    ax.scatter(X[y == 0, 0], X[y == 0, 1],
               c="#B2B9C1", marker="s",
               edgecolor="k", s=30, label="Classe 0")
    ax.scatter(X[y == 1, 0], X[y == 1, 1],
               c="#FF6B35", marker="o",
               edgecolor="k", s=30, label="Classe 1")
    ax.set_xlim(x_lim)
    ax.set_ylim(y_lim)
    ax.set_aspect("equal")
    ax.set_xlabel("$x_1$")
    ax.set_ylabel("$x_2$")
    ax.set_title(title)
    ax.legend(loc="upper center", ncol=3,
              frameon=False,
              bbox_to_anchor=(0.5, -0.12))
x_lim = (X[:, 0].min() - 0.3, X[:, 0].max() + 0.3)
y_lim = (X[:, 1].min() - 0.3, X[:, 1].max() + 0.3)
fig, ax = plt.subplots(figsize=(6, 6))
plot_decision_boundary(svm_linear, X_train, y_train,
    ax, "SVM Linear (C=1.0)", x_lim, y_lim)
plt.show()
```

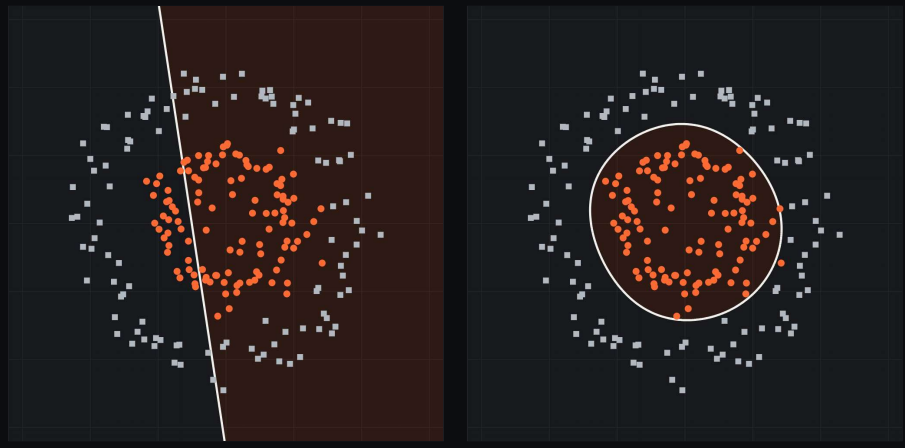
11 PIPELINE RBF · 12 FIT · 13 AVALIAR · 14 COMPARAR

```
svm_rbf = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", C=1.0, gamma="scale"))
])
svm_rbf.fit(X_train, y_train)

X_train_scaled = svm_rbf.named_steps["scaler"].transform(X_train)
print("gamma efetivo:",
      1 / (X_train_scaled.shape[1] * X_train_scaled.var()))

pred_rbf = svm_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
cm_rbf = confusion_matrix(y_test, pred_rbf)
print(f"Acurácia (RBF): {acc_rbf:.3f}")
print(cm_rbf)

fig, axes = plt.subplots(1, 2, figsize=(13, 6))
plot_decision_boundary(svm_linear, X_train, y_train, axes[0],
    f"SVM Linear | teste {acc_linear:.3f}", x_lim, y_lim)
plot_decision_boundary(svm_rbf, X_train, y_train, axes[1],
    f"SVM RBF | teste {acc_rbf:.3f}", x_lim, y_lim)
plt.tight_layout()
plt.show()
```



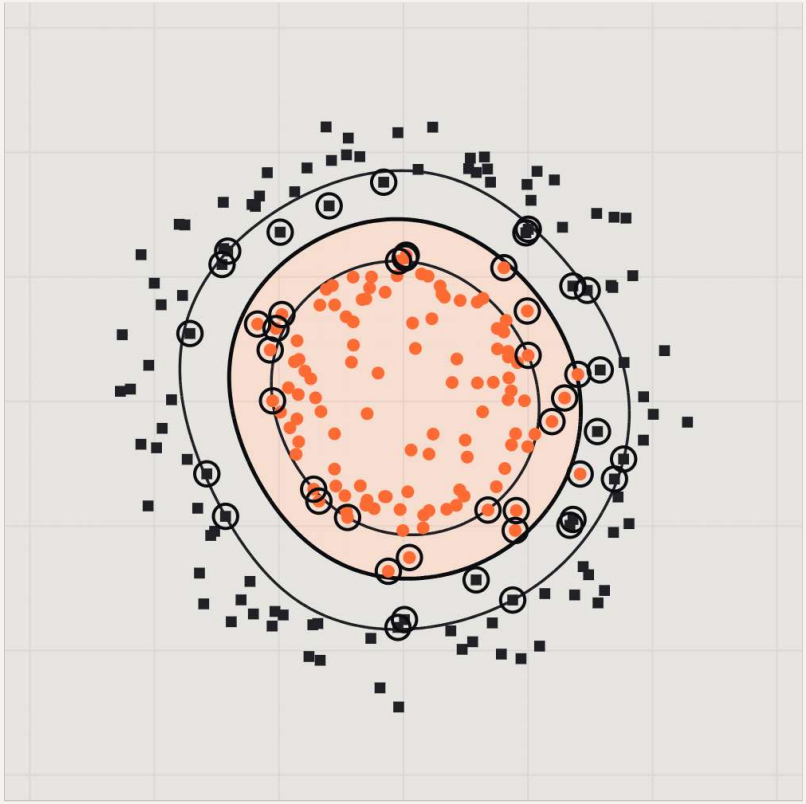
Linear RBF

Mesmos limites, mesmas amostras, mesmas cores. As fronteiras vêm das previsões reais dos pipelines.

Mesmos dados e split do notebook. Execução de referência (scikit-learn 1.4.2): teste linear 0,493 · RBF 1,000 · gamma efetivo 0,5.

15 · VETORES DE SUPORTE NAS COORDENADAS CORRETAS

```
svc_rbf = svm_rbf.named_steps["svc"]
idx_sv = svc_rbf.support_
print("Total de vetores de suporte:", len(idx_sv))
print("Por classe:", svc_rbf.n_support_)
# índices valem para X_train (mesma ordem)
sv_original = X_train[idx_sv]
# support_vectors_ está ESCALONADO
scaler = svm_rbf.named_steps["scaler"]
sv_reconstruido = scaler.inverse_transform(
    svc_rbf.support_vectors_)
print(np.allclose(sv_original, sv_reconstruido))
fig, ax = plt.subplots(figsize=(6, 6))
plot_decision_boundary(svm_rbf, X_train, y_train, ax,
    "SVM RBF | vetores de suporte", x_lim, y_lim)
ax.scatter(sv_original[:, 0], sv_original[:, 1],
           s=160, facecolors="none",
           edgecolors="#0C0D10", linewidths=1.5,
           label="Vetores de suporte")
ax.legend(loc="upper center", ncol=3,
          frameon=False,
          bbox_to_anchor=(0.5, -0.12))
plt.show()
```



Não desenhe support_vectors_ direto sobre os dados originais: use X_train[support_] ou inverse_transform.

16 · EXPERIMENTO COM C (gamma = 1.0 fixo)

```
valores_C = [0.01, 1, 100]
modelos_C = {}
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, C in zip(axes, valores_C):
    modelo = Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(kernel="rbf", C=C,
                    gamma=1.0)),
    ])
    modelo.fit(X_train, y_train)
    modelos_C[C] = modelo
    acc_tr = accuracy_score(
        y_train, modelo.predict(X_train))
    acc_te = accuracy_score(
        y_test, modelo.predict(X_test))
    plot_decision_boundary(
        modelo, X_train, y_train, ax,
        f"C={C} | treino {acc_tr:.3f}"
        f" | teste {acc_te:.3f}",
        x_lim, y_lim)
plt.tight_layout()
plt.show()
```

17 · EXPERIMENTO COM GAMMA (C = 1.0 fixo)

```
valores_gamma = [0.1, 1, 10]
modelos_gamma = {}
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for ax, g in zip(axes, valores_gamma):
    modelo = Pipeline([
        ("scaler", StandardScaler()),
        ("svc", SVC(kernel="rbf", C=1.0,
                    gamma=g)),
    ])
    modelo.fit(X_train, y_train)
    modelos_gamma[g] = modelo
    acc_tr = accuracy_score(
        y_train, modelo.predict(X_train))
    acc_te = accuracy_score(
        y_test, modelo.predict(X_test))
    plot_decision_boundary(
        modelo, X_train, y_train, ax,
        f"gamma={g} | treino {acc_tr:.3f}"
        f" | teste {acc_te:.3f}",
        x_lim, y_lim)
plt.tight_layout()
plt.show()
```

18 · TABELA COMPARATIVA (VALORES DA EXECUÇÃO)

```
def resumo(nome, modelo):
    svc = modelo.named_steps["svc"]
    g = "—" if svc.kernel == "linear" else svc.gamma
    return [nome, svc.kernel, svc.C, g,
        accuracy_score(y_train, modelo.predict(X_train)),
        accuracy_score(y_test, modelo.predict(X_test)),
        int(svc.n_support_.sum())]

linhas = [resumo("Linear", svm_linear),
          resumo("RBF scale", svm_rbf)]
linhas += [resumo(f"RBF C={c}", m)
           for c, m in modelos_C.items()]
linhas += [resumo(f"RBF gamma={g}", m)
           for g, m in modelos_gamma.items()]

print(f"{'Modelo':<15}{'Kernel':<8}{'C':>7}"
      f"{'gamma':>8}{'Treino':>9}{'Teste':>8}{'VS':>6}")
for nome, k, c, g, tr, te, nv in linhas:
    print(f"{nome:<15}{k:<8}{c:>7}{str(g):>8}"
          f"{tr:>9.3f}{te:>8.3f}{nv:>6}")
```

19 · CONCLUSÃO (CÉLULA MARKDOWN)

Linear: uma reta não separa disco e anel.

RBF: similaridade por distância permite uma fronteira fechada.

C: custo das violações, equilíbrio entre margem e ajuste.

Gamma: alcance da influência de cada vetor de suporte.

Escalonamento: necessário para distâncias; dentro do pipeline.

`Restart Kernel → Run All`

Erros conceituais e correções

01	SVM é regressão logística	Mesma forma de fronteira linear; objetivos diferentes (hinge vs log-loss). SVC não dá probabilidade.
02	Qualquer reta separadora é ótima	A SVM escolhe a de margem máxima (ou o melhor equilíbrio margem × violações).
03	Margem = distância entre centroides	Margem = distância da fronteira às amostras mais próximas.
04	Todos os pontos são vetores de suporte	Só os com $\alpha_i > 0$. Confira n_support_.
05	Confundir C e gamma	C: custo das violações (qualquer kernel). γ: alcance do kernel RBF.
06	Gamma só existe na SVM	O kernel RBF aparece em outros métodos (Kernel PCA, Kernel Ridge, rbf_kernel).
07	RBF = transformação 3D explícita	$z = x_1^2 + x_2^2$ foi didática. O RBF corresponde a um espaço de dimensão infinita.
08	Acurácia de treino prova generalização	Use validação para escolher e teste para estimar.

Erros de programação e correções

09	Esquecer o escalonamento	Pipeline com StandardScaler antes da SVC.
10	Escalonar antes de dividir	train_test_split primeiro; scaler dentro do pipeline.
11	Ler predict() como probabilidade	predict = rótulo; decision_function = valor real. Probabilidade: SVC(probability=True).
12	Grade em coordenadas erradas	Grade nas coordenadas originais; o pipeline escalona.
13	Plotar support_vectors_ direto	Estão escalonados: X_train[support_] ou inverse_transform.
14	Comparar com divisões diferentes	Mesmo X_train / X_test para todos os modelos.
15	Função auxiliar não definida	Defina plot_decision_boundary no notebook, antes do uso.
16	Nomes diferentes entre slides e notebook	X_train, svm_linear, svm_rbf, pred_linear, acc_linear...

DESAFIO FINAL · make_moons(n_samples=300, noise=0.20, random_state=42)

Investigando fronteiras de decisão com SVM

01 Criar dataset com make_moons
02 Visualizar os dados
03 Separar treino/teste
04 Pipeline com StandardScaler
05 Treinar SVM Linear
06 Treinar SVM RBF
07 Comparar os modelos
08 Calcular acurácia
09 Matrizes de confusão
10 Visualizar fronteiras
11 Testar valores de C
12 Testar valores de gamma
13 Interpretar os resultados

ENTREGAS

Notebook executável (Restart → Run All)
Gráficos das fronteiras
Métricas
Comparação dos hiperparâmetros
Explicação das diferenças
Conclusão

`Desafio_SVM_make_moons_ALUNOS.ipynb`
`Extensão opcional: GridSearchCV`

Leitura complementar

PRÁTICA

scikit-learn User Guide, seção Support Vector Machines; documentação de sklearn.svm.SVC; exemplo RBF SVM parameters.

LIVROS

James, Witten, Hastie, Tibshirani. An Introduction to Statistical Learning, cap. 9. Hastie, Tibshirani, Friedman. The Elements of Statistical Learning, cap. 12.

ARTIGOS FUNDADORES

Boser, Guyon, Vapnik (1992). A training algorithm for optimal margin classifiers. Cortes, Vapnik (1995). Support-vector networks.

APROFUNDAMENTO

Platt (1998). Sequential Minimal Optimization. Schölkopf, Smola (2002). Learning with Kernels.

Hiperplano → Margem → Vetores de suporte → Soft margin → C → Kernel → RBF → Gamma → Fronteira de decisão